Java OOP

Lecturer: Muksimova Shakhnoza

Java - What is OOP?

OOP stands for **Object-Oriented Programming**

Procedural programming is about writing procedures or methods that perform operations on the data, while object-oriented programming is about creating objects that contain both data and methods.

Object-oriented programming has several advantages over procedural programming:

- OOP is faster and easier to execute
- OOP provides a clear structure for the programs
- OOP helps to keep the Java code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug
- OOP makes it possible to create full reusable applications with less code and shorter development time

Java - What are Classes and Objects?

- Classes and objects are the two main aspects of object-oriented programming.

class	objects
Fruit	Apple
	Banana
	Mango

Another example:

class	objects
Car	Volvo
	Audi
	Toyota

Java Classes/Objects

Java is an **object-oriented programming** language.

- Everything in Java is associated with classes and objects, along with its attributes and methods. For example: in real life, a car is an object. The car has **attributes**, such as weight and color, and **methods**, such as drive and brake.
- A Class is like an object constructor, or a "blueprint" for creating objects.

Create a Class

To create a class, use the keyword class.

In this example, we create a class named "**Main**" with a variable **x**:

Main.java

```
public class Main {
int x = 5;
}
```

Create an Object

- In Java, an object is created from a class. After defining a class, you can create objects from it using the new keyword:

Create an object called "myObj" and print the value of x:

```
public class Main {
  int x = 5;
  public static void main(String[] args) {
    Main myObj = new Main();
    System.out.println(myObj.x);
  }
}
```

Multiple Objects

- Create two objects of Main:

```
public class Main {
  int x = 5;

  public static void main(String[] args) {
    Main myObj1 = new Main();  // Object 1
    Main myObj2 = new Main();  // Object 2
    System.out.println(myObj1.x);
    System.out.println(myObj2.x);
  }
}
```

Using Multiple Classes

You can also create an object of a class and access it in another class. This is often used for better organization of classes (one class has all the attributes and methods, while the other class holds the main() method (code to be executed)).

Remember that the name of the java file should match the class name. In this example, we have created two files in the same directory/folder:

- Main.java
- Second.java

Main.java

```
public class Main {
  int x = 5;
}
```

Second.java

```
class Second {
  public static void main(String[] args) {
    Main myObj = new Main();
    System.out.println(myObj.x);
  }
}
```

Output:5

Java Class Attributes

We used the term "**variable**" for x in the example (as shown below).

In Java, variables declared inside a class are called "**attributes**".

Create a class called "Main" with two attributes: x and y:

```
public class Main {
  int x = 5;
  int y = 3;
}
```

Accessing Attributes

We can access attributes by creating an object of the class, and by using the dot syntax (.):

The following example will create an object of the Main class, with the name myObj. We use the **x** attribute on the object to print its value:

Create an object called "myObj" and print the value of x:

```
public class Main {
  int x = 5;

  public static void main(String[] args) {
    Main myObj = new Main();
    System.out.println(myObj.x);
  }
}
```

Modify Attributes

Set the value of x to 40:

```
public class Main {
  int x;

  public static void main(String[] args) {
    Main myObj = new Main();
    myObj.x = 40;
    System.out.println(myObj.x);
  }
}
```

Change the Value

Change the value of x to 25:

```
public class Main {
  int x = 10;

  public static void main(String[] args) {
    Main myObj = new Main();
    myObj.x = 25; // x is now 25
    System.out.println(myObj.x);
  }
}
```

Java Class Methods

Create a method named myMethod() in Main:

```
public class Main {
  static void myMethod() {
    System.out.println("Hello World!");
  }
}
```

myMethod() prints a text (the action), when it is **called**. To call a method, write the method's name followed by two parentheses **()** and a semicolon**;**

Example

Inside main, call myMethod():

```
public class Main {
  static void myMethod() {
    System.out.println("Hello World!");
  }

  public static void main(String[] args) {
    myMethod();
  }
}

// Outputs "Hello World!"
```

Access Methods With an Object

- Create a Car object named myCar. Call the fullThrottle() and speed( ) methods on the myCar object, and run the program:

```java
// Create a Main class
public class Main {

  // Create a fullThrottle() method
  public void fullThrottle() {
    System.out.println("The car is going as fast as it can!");
  }

  // Create a speed() method and add a parameter
  public void speed(int maxSpeed) {
    System.out.println("Max speed is: " + maxSpeed);
  }

  // Inside main, call the methods on the myCar object
  public static void main(String[] args) {
    Main myCar = new Main();   // Create a myCar object
    myCar.fullThrottle();      // Call the fullThrottle() method
    myCar.speed(200);          // Call the speed() method
  }
}
```

Task

Inside the Main class, complete the following steps:

- Create a String variable named city and set it to "**London**"
- Create an **int** variable named population and set it to **9000000**
- Inside **main()**, create an object of **Main** named myObj
- Print **city** and population using **myObj**

Java Constructors

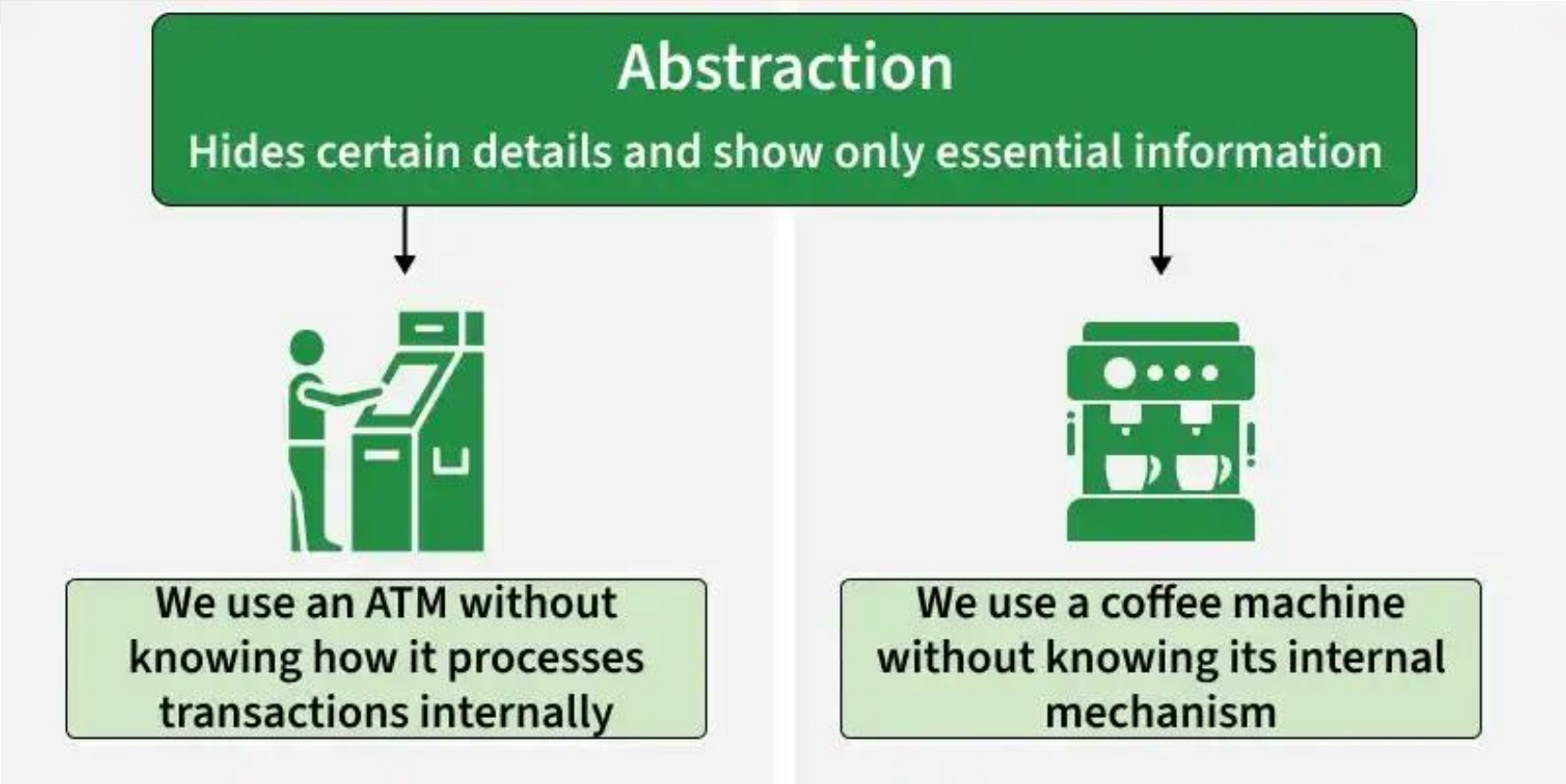
- A constructor in Java is a **special method** that is used to initialize objects.
- The constructor is called when an object of a class is created.
- It can be used to set initial values for object attributes:

```java
// Create a Main class
public class Main {
  int x;  // Create a class attribute

  // Create a class constructor for the Main class
  public Main() {
    x = 5;  // Set the initial value for the class attribute x
  }

  public static void main(String[] args) {
    Main myObj = new Main(); // Create an object of class Main (This will call the constructor)
    System.out.println(myObj.x); // Print the value of x
  }
}
```

Abstraction

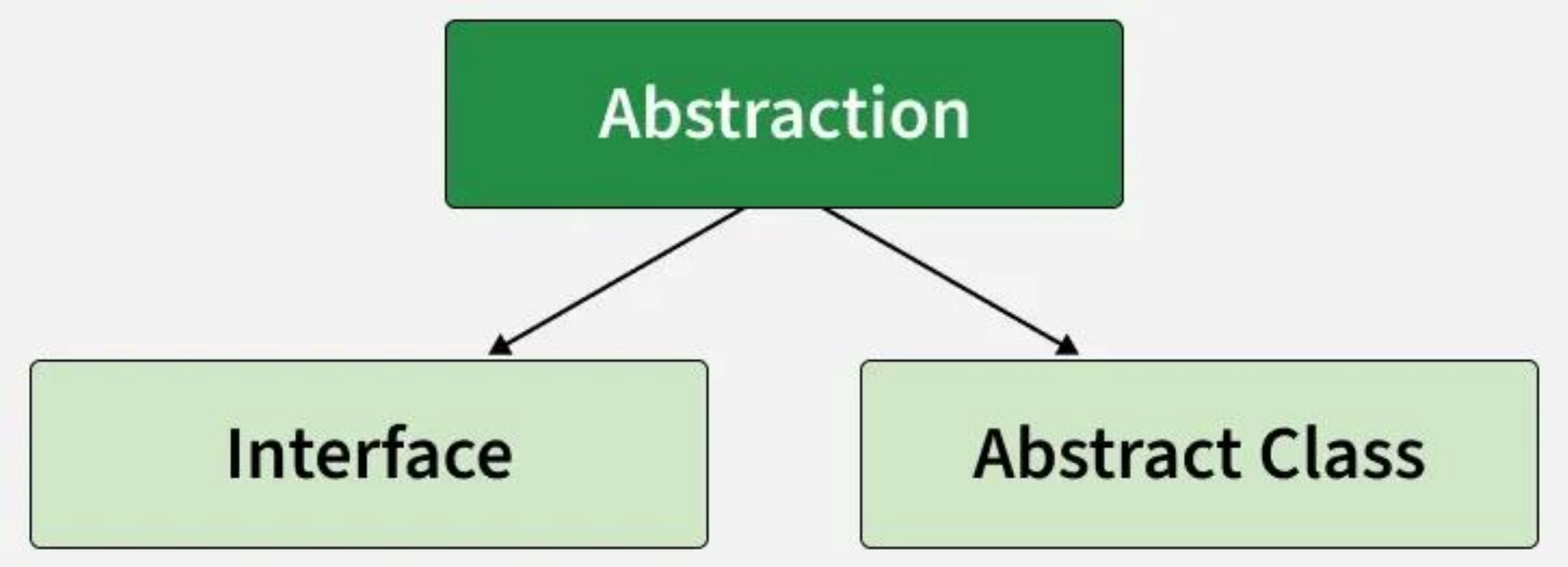
- Abstraction in Java is the process of hiding implementation details and showing only the essential features of an object. It helps users focus on what an object does rather than how it does it.
- **Hides complexity:** Internal implementation details are hidden from the user.
- **Improves maintainability:** Changes in implementation do not affect the user code.
- **Enhances flexibility:** Supports loose coupling through abstract classes and interfaces.

An ATM or a coffee machine represents abstraction, where the user interacts with simple operations while the internal working and implementation details remain hidden.

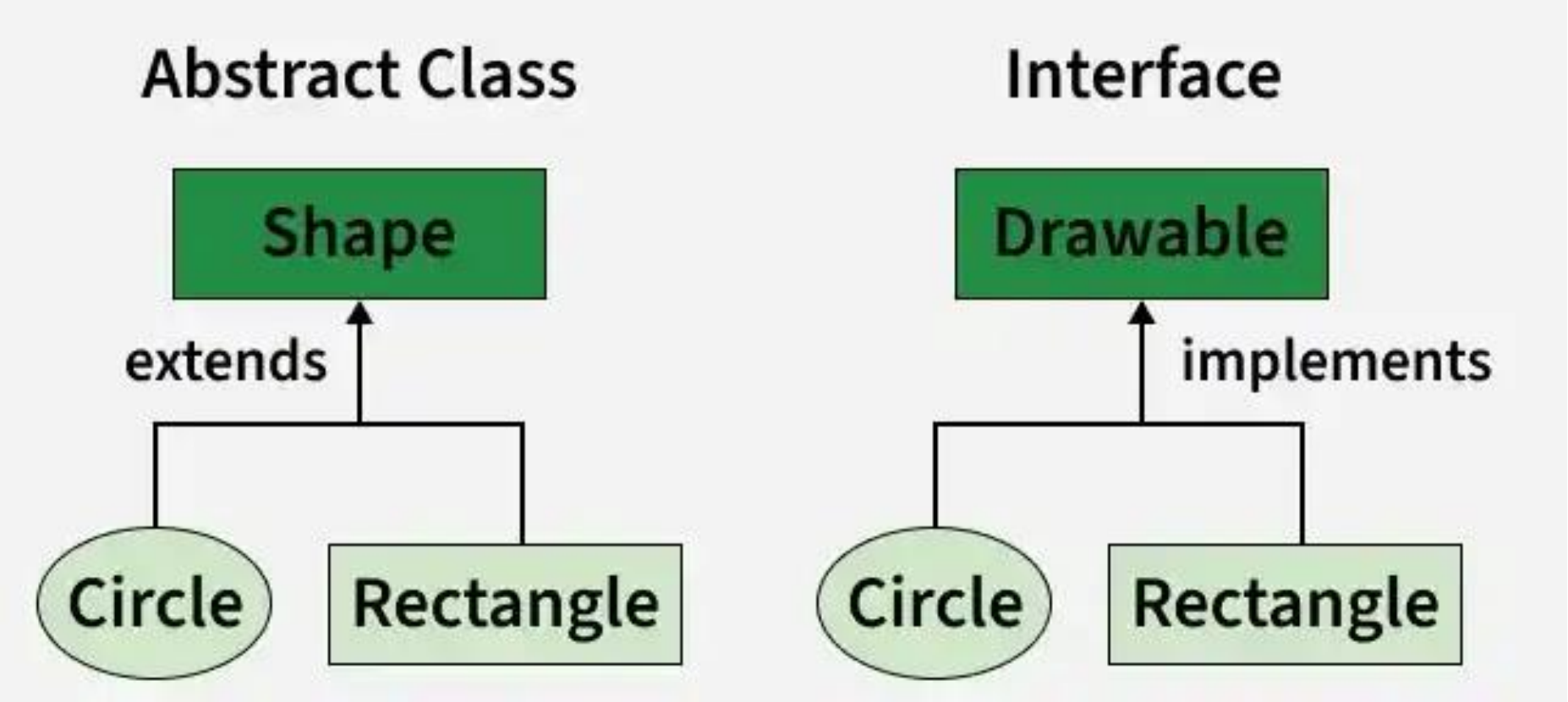


How to Achieve Abstraction

- It is achieved in Java using abstract classes and interfaces.
- Interfaces can provide 100% abstraction, while abstract classes provide partial abstraction.

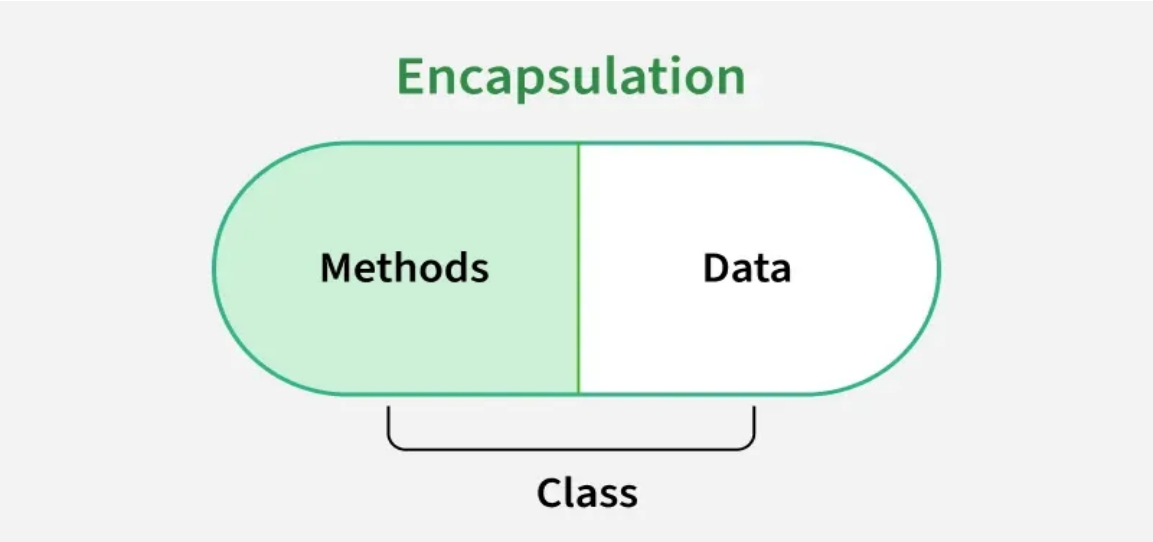


Abstract class defines a common base (Shape) using inheritance, while an interface (Drawable) defines behavior implemented by multiple classes.



Encapsulation

- Encapsulation is the process of wrapping data and methods into a single unit, usually a class, and restricting direct access to the data. It acts as a protective shield that prevents data from being accessed directly from outside the class.
- Data members are hidden using the private access modifier.
- Access to data is provided through public getter and setter methods.
- It improves data security, maintainability, and controlled access.

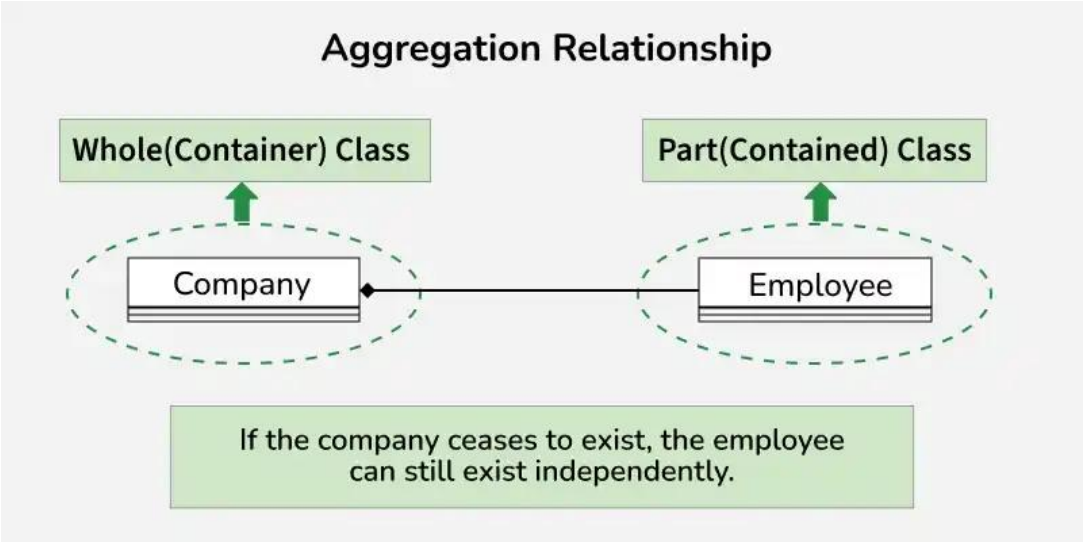


Association

- Association is an OOP concept that defines a relationship between two or more classes that are connected to each other. It represents how objects interact with each other and communicate. In association, objects of one class are related to objects of another class, but they can exist independently.
- Represents a relationship between objects
- Does not imply ownership
- Objects are independent of each other

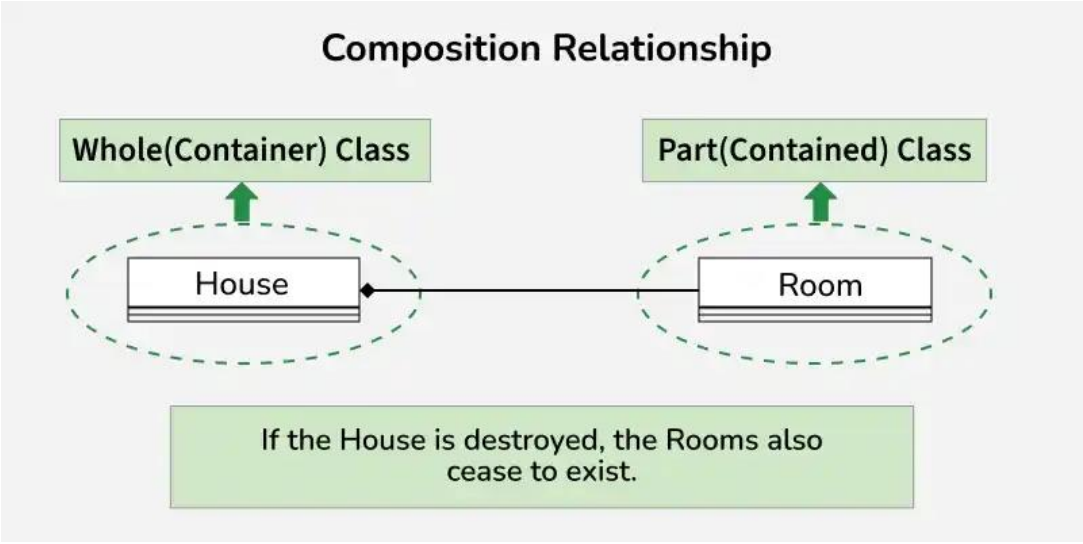
Types of Association

- **1. Aggregation (Weak Association)**
- Aggregation represents a “has-a” relationship where one class contains a reference to another class, but both can exist independently.
- It is a weak relationship
- Objects have independent lifecycles
- One object can exist without the other
- **Ex:** A Company has Employees, but employees can exist independently even if the company no longer exists.



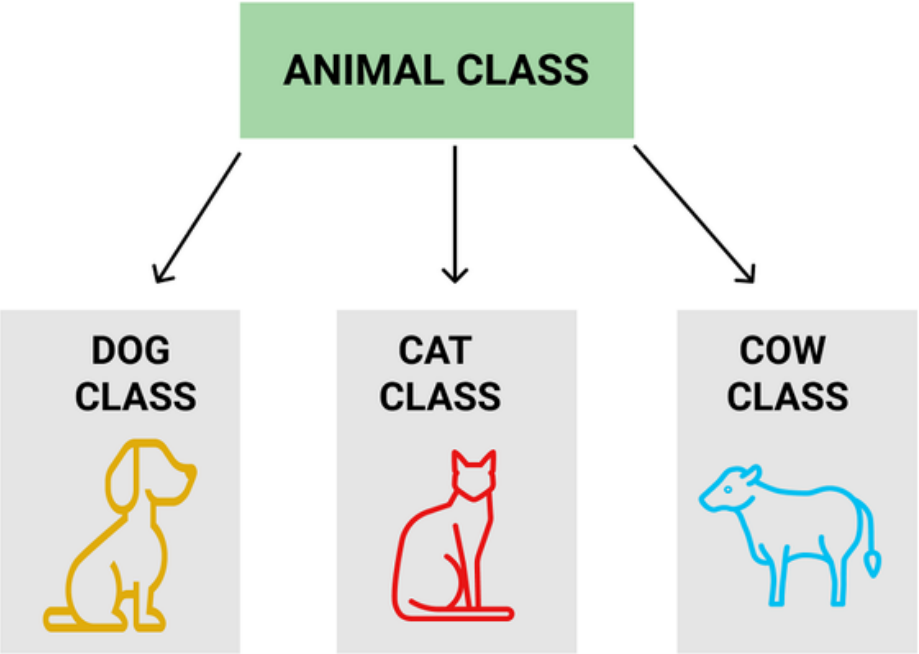
2. Composition (Strong Association)

- Composition is a strong form of association where one class owns another class. If the parent object is destroyed, the child object also gets destroyed.
- It is a strong relationship
- Objects have dependent lifecycles
- Child object cannot exist without the parent
- **Ex:** A House is composed of Rooms, and if the house is destroyed, the rooms cannot exist independently.



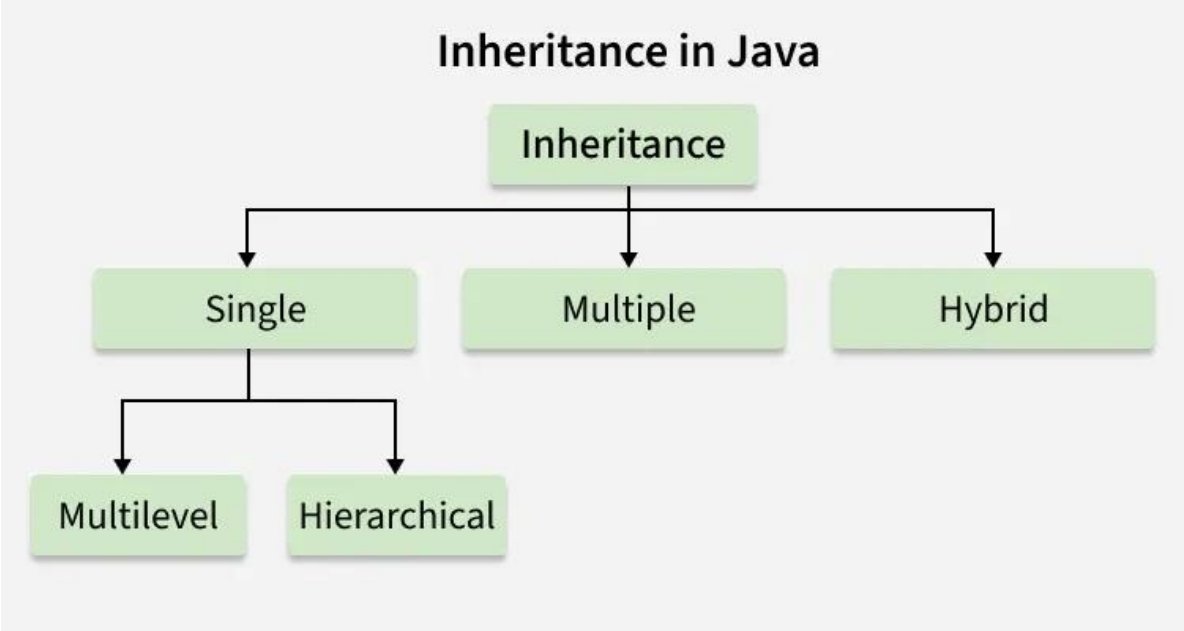
Inheritance

- Inheritance is a core OOP concept in Java that allows one class to acquire the fields and methods of another class using the extends keyword. It represents an “is-a” relationship between classes.
- The class being inherited is called the superclass, and the inheriting class is the subclass.
- A subclass can use existing features of the superclass and also add its own.
- Inheritance promotes code reusability and reduces redundancy.
- **Example**: Dog, Cat, Cow can be Derived Class of Animal Base Class.



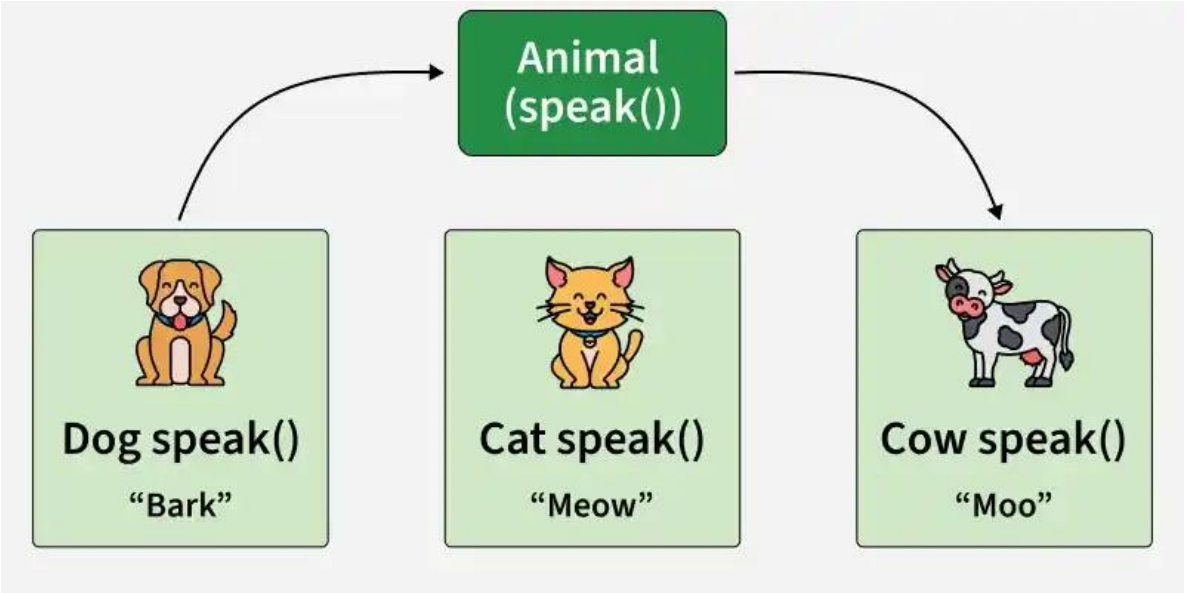
Types of Inheritance

- Java supports the following types of inheritance:
- **Single Inheritance:** One subclass inherits from one superclass.
- **Multilevel Inheritance:** A class is derived from another derived class, forming a chain.
- **Hierarchical Inheritance:** Multiple subclasses inherit from a single superclass.
- **Multiple Inheritance (through Interface):** A class inherits from multiple interfaces since Java does not support multiple inheritance using classes.
- **Hybrid Inheritance (through Interface):** A combination of two or more types of inheritance, achievable using interfaces.



Polymorphism

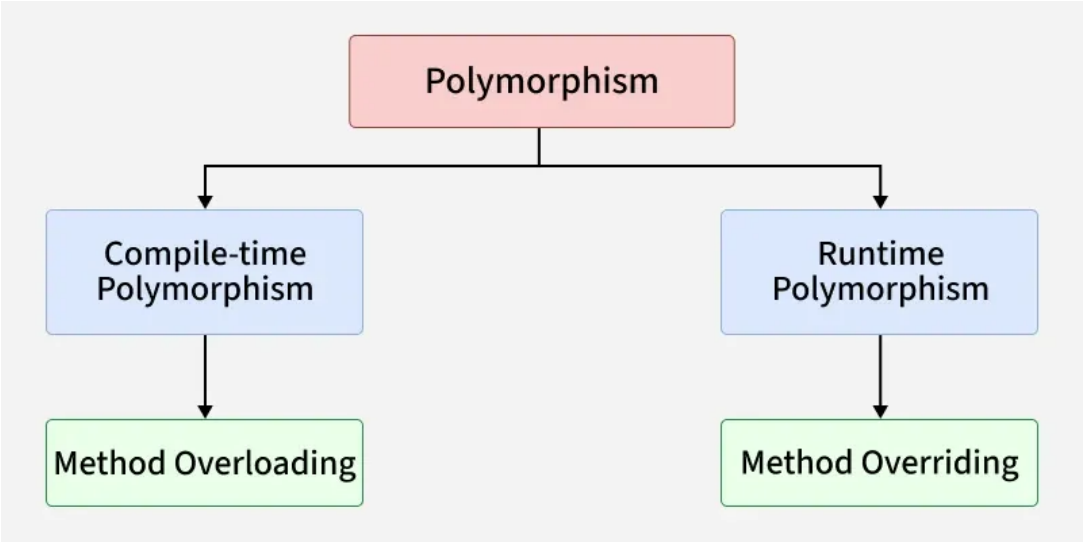
- Polymorphism means “many forms”, where a single entity can behave differently in different situations. In Java, it allows the same method or object to show different behavior based on context.
- Same method, different behavior depending on the object
- Achieved through method overloading and method overriding
- **Example:** Different animals represent polymorphism, where the same method speak() produces different outputs like Bark, Meow, and Moo depending on the object.



Types of Polymorphism

- **Compile-time Polymorphism(**Method Overloading**) :**Achieved when multiple methods have the same name but different parameters. The method call is resolved at compile time.

- **Runtime Polymorphism (**Method Overriding **):** Achieved when a subclass provides a specific implementation of a method already defined in its superclass. The method call is resolved at runtime based on the object



Advantage of OOP

- **Code reusability:** Classes and objects allow reuse of existing code, reducing duplication and improving efficiency.
- **Better structure and maintainability:** Programs are organized into logical units, making code easier to understand, debug, and maintain.
- **Supports DRY principle:** Common functionality is written once and reused, leading to cleaner and more maintainable code.
- **Faster development:** Modular and reusable components help in quicker and scalable application development.
-

Disadvantages of OOP

- **Steep learning curve:** Concepts like classes, objects, inheritance, and polymorphism can be difficult for beginners.
- **Overhead for small programs:** OOP may require more code and structure than necessary for simple applications.
- **Debugging complexity:** Code spread across multiple classes and layers can make debugging more time-consuming.
- **Higher memory usage:** Creating many objects can consume more memory compared to procedural programs.